When does a manufacturability term change what evolution selects? Measurement, repair and a computable screening bound, with the ForgeCraft platform

Zhenxin Liu[1]*, Open Researcher and Contributor Identifier (ORCID) 0009-0005-0773-647X

[1] Beijing Jiaotong University, Beijing, China

** Corresponding author. E-mail: liuzhenxin0611@qq.com*

Abstract

Background. Morphology–control co-design searches simulated link-and-joint assemblies, so the part geometry, tolerances and clearances that decide whether a design can be built never enter the objective. Adding a manufacturability term is the usual remedy, and its effectiveness is usually assumed rather than measured. **Aim.** We ask when such a term can change what selection keeps, and we answer it with measurement rather than with a new search algorithm. The vehicle is ForgeCraft, an open-source Python platform that couples declarative part catalogues, graph neural network (GNN) co-evolution with proximal policy optimisation (PPO) controllers, MuJoCo evaluation, and an export layer emitting tessellated assemblies in the Standard for the Exchange of Product model data (STEP), application protocol 242 (AP242), drawings in Drawing Exchange Format (DXF), interference reports and bills of materials. **Method.** We ablate the term counterfactually within single runs, on two instrument builds, and derive a bound on the reordering any weight can produce. **Results.** The shipped build (tag v0.4.0) could not act on the term at all: the score's range lay inside [0.80, 1.00] and its weight was never read from the task file, so every run silently used 0.05. After repair the score's mean within-catalogue range rises from 0.024 to 0.225 and the term does reorder individuals, but only a minority of comparisons: at the deployed weight the discordant count has a median of 1 of 120 pairs on one catalogue and 3 of 120 on the other, non-zero in 7 and 10 of 13 seeds, and top-five membership changes in 1 and 6 of 13 seeds. A pair can reverse only when its fitness gap is smaller than the term's contribution. The fraction of such pairs is computable in one generation and bounds reordering from above; its median is 12–29% for one catalogue and 16–68% for the other over a fortyfold weight sweep, which is why the same nominal weight behaves differently in each. We condense the bound into a five-case screening protocol that decides, from one generation and without an evolutionary run, whether a given weight is worth a seed budget. Whether crossing the bound improves populations is measurable, and it does not: over thirteen seeds per arm the mean manufacturability is 0.799 at a weight of 0.50 and 0.771 at 1.00, against 0.810 at the shipped weight, while the count of individuals below the 0.8 threshold rises from 88 to 93 and 112 of 208. The realised diagnostic ratio saturates near 0.64 instead of doubling with the weight, because the population's fitness spread grows sixfold while its manufacturability spread grows by half. At the lower dose the power calculation asks for 6 seeds at a difference of 0.10 and 57 at 0.03. **Limitation.** No evolved design has been fabricated, and the export layer reveals defects the objective never sees: every final individual contains colliding part pairs. Both measurements come from this framework's generation of the codebase and two of its catalogues, so the audit is stated in terms of the objective and the population rather than as an established property of other frameworks.

Keywords: evolutionary robotics · morphology optimisation · design for additive manufacturing · STEP AP242 · reproducibility · open-source software

1 Introduction

The body of a robot largely determines what that robot can do, and in practice the body is normally fixed by human designers before any controller is learned. Co-design — the joint optimisation of morphology and control — has been studied since the first work on evolving virtual creatures [1] and has been revisited repeatedly with modern machinery: graph-grammar design spaces [2], learned transform-and-control policies [4], self-assembling modular morphologies [5], and quality-diversity search over collections of designs [6, 7].

A persistent limitation runs through this literature. Evaluated designs are almost always *simulated* bodies: abstract link-and-joint assemblies whose geometric realisation is left to the reader. An evolved morphology cannot be built, however, without answering questions the simulation never asks. Which abstract links correspond to printable parts? Do they fit a build volume? Are the walls thick enough? Do adjacent parts collide once real bearings,

fasteners and clearances are introduced? Can the joint be assembled at all, and what does the bill of materials cost? These are not cosmetic concerns: they decide whether an evolved design leaves the computer. When manufacturability is absent from the objective, search is free to exploit shapes that are optimal in simulation and unbuildable in reality, and a post-hoc buildability filter then discards exactly the designs the search converged on.

Two further gaps compound the problem. **Reproducibility:** morphology-evolution results are frequently reported for bespoke codebases that cannot be re-run, so comparisons between methods remain approximate. **Fragmentation:** the capabilities required for a complete study — morphology generation, physics evaluation, control learning, geometric modelling and manufacturing preparation — are normally distributed across separate tools with incompatible data models, so every stage boundary is crossed by hand.

Contribution. The measurement is the contribution; the platform is what made it possible. (i) A counterfactual method that removes an objective term *within* a single run, so that no re-evaluation is needed and unpaired worker seeds cannot contaminate the comparison. (ii) A two-build measurement showing that the shipped release could not act on its manufacturing term at all, and identifying the two instrumentation defects responsible — a threshold-triggered score with no gradient, and a weight lookup that read the wrong configuration namespace, so every task silently used a hard-coded fallback. (iii) A bound, proved in Section 7.5 and verified on every cell of two sweeps, on how many pairs of individuals any weight can reorder, together with the single-generation quantity that makes the bound usable before a long run is spent: the fraction of pairs whose fitness gap is smaller than the term's contribution. (iv) A replication budget — 6 to 105 seeds per arm, for differences of 0.10 down to 0.03 — that states what evidence a population-level claim would require, a dose-response test at two weights showing that the budget buys an answer rather than the answer hoped for (Section 7.7), and a five-case screening protocol (Section 7.10) that decides from one generation whether a given weight is worth spending that budget, with each case instantiated by a row of our own sweep. The platform itself contributes an explicit parametric representation of evolved bodies as mechanical parts sharing parameters with the bodies that were evaluated, and an export chain that converts such a body into the deliverables a fabricator can consume. We also report, in Section 8, the defects and inconsistencies found in our own release, because a framework making manufacturing claims is only as useful as its account of where those claims currently stop.

A three-step audit, stated generally. The two defects and the bound reduce to three checks that any framework with a scalarised objective can run, and we set them out here as the paper's method rather than as a report on one codebase. *Range check*: compute the term's own values across a sample of the population; if its increments are not larger than the evaluation noise, no weight can act on them. *Resolution check*: log the weight that the objective actually receives at the call site rather than the value the configuration nominally sets, because a silent fallback is invisible in the source and visible in one line of output. *At-risk screen*: compute r from the fitness spread, the term's spread and the weight on a single generation; below one, the term cannot separate even a near-tie pair, and the number of pairs it can still reorder forecasts the size of the effect a run can produce. ForgeCraft is the instance in which the first two checks fail and the third is needed; Sections 7.5 to 7.10 give each check a proof, a measurement and a decision rule, and the order matters, because the screen is only meaningful once the first two have passed.

2 Related work and the position of this work

Modern co-design systems encode a body as a graph and optimise it with evolutionary search while a learned controller is fitted to each candidate; ForgeCraft follows that template and differs in the treatment of the output. The body graph is not only a simulation input but also the specification from which concrete mechanical geometry is generated, so the artefact evaluated and the artefact exported derive from one shared parameter set (Section 3.4).

Three platforms are close enough in ambition to warrant direct comparison, and Table 1 summarises it. **Evolution Gym** [8] is a large-scale benchmark for co-optimising the design and control of soft robots, in which a body is a 2D lattice of material cells simulated as a mass-spring system. **ARIEL** (https://ci-group.github.io/ariel/), the successor to the Revolve simulator [9], covers genome encoding, evolution, MuJoCo simulation and result analysis for modular robots. **JBotEvolver** [10] is a long-established Java platform for evolutionary robotics research and teaching, built around a 2D differential-drive engine.

All three are strong at what they set out to do, and ForgeCraft integrates the same class of components rather than reimplementing them. The relevant difference is the terminal stage: in the released feature sets of all three, the pipeline ends once a simulation result has been produced and logged, and none describes a route from an evolved body to manufacturing deliverables. For Evolution Gym this follows from the representation itself — a 2D voxel matrix has no corresponding fabrication output. For ARIEL and JBotEvolver, whose robots are built from physical

modules or wheeled bases, the morphologies are realisable in hardware by construction, but the mapping from an evolved graph to machine-readable part geometry, drawings and bills of materials is not part of the framework.

Design for additive manufacturing [11] and design for manufacture and assembly [12] supply the criteria a buildable part must satisfy — minimum feature size, wall thickness, build-volume fit, support and orientation, assembly complexity — and product data exchange between computer-aided design (CAD) systems is standardised in International Organization for Standardization (ISO) 10303, whose AP242 part [13] covers managed model-based 3D engineering, with general tolerances given by ISO 2768-1 [14]. This body of work is mature, but it does not address a design generated by a search free to exploit any unmodelled degree of freedom — precisely the case ForgeCraft is built for.

Where this paper stands relative to that literature. The limitations of a weighted sum are not in dispute. For a non-convex front some Pareto-optimal points are unreachable by any positive weight, and the ordering a weight induces can change with the weight itself [15, 16, 17, 18]. Multi-objective evolutionary research has consequently treated the scalarisation as a design decision, and has built the machinery for reporting what a front contains rather than what a scalar ranks [19, 20]. Robust optimisation perturbs an objective that it assumes to be correctly defined, and asks how sensitive the optimum is to the perturbation [21, 22]. Design for additive manufacturing supplies the geometric criteria — feature size, wall thickness, orientation, support, assembly — that a buildability term is meant to encode [23, 24, 25]. The search-side alternatives to a weighted sum are equally developed: dominance archives and elite maps [26, 27]; and on the geometry side, the parameterisation of parts for fabrication is itself an established subject [28].

None of these lines asks the question this paper asks, because each begins from an objective it assumes to be operating. One asks how faithfully a scalar ordering represents a front; another how sensitive a located optimum is to perturbation; the last which criteria a buildable part must satisfy. We begin from a term that is declared in the configuration, reported in every log line and nominally active, and ask whether it can act at all. The answer is not another statement that weighted sums are fragile. It is a screening quantity — the fraction of pairs whose fitness gap is smaller than the term's contribution — computable from one generation, before any run is spent, and bounding the reordering any weight can produce on the population in front of the reader (Section 7.5); an empirical account of two silent failures in a working system, in which the term was inert for reasons unrelated to the mathematics of scalarisation, one a score with no gradient and one a weight never read from the configuration (Sections 7.2–7.3); and a budget, with a dose-response test attached, that turns "how many seeds would this need" into a number and separates the reordering a weight buys from the improvement a reader would assume it buys (Section 7.7). The design criteria of [23, 24, 25] are taken as given; what we measure is what a search does with them once they enter a scalar objective, on two catalogues of one framework.

Platform	Morphology representation	Co-design search	Physics	Manufacturing deliverables	Reference
Evolution Gym	2D lattice of material voxels (soft / rigid / actuator)	Yes — design and control co-optimisation; task suite and baseline algorithms	Own C++ mass-spring engine	None in the released benchmark	Bhatia et al. [8]
ARIEL (formerly Revolve)	Modular robot graph decoded from compositional pattern-producing network (CPPN) / tree / vector genotypes	Yes — body–brain co-design with evolutionary-algorithm (EA) operators	MuJoCo	None in the published feature set	[9]
JBotEvolver	2D differential-drive kinematics; controller evolution on a fixed or scripted body	Controller evolution; body variation is not the focus	Own 2D engine	None	[10]
ForgeCraft	Typed graph of parametric parts drawn from declarative YAML catalogues	Yes — NSGA fronts and a multi-dimensional archive of phenotypic elites over bodies; PPO controller conditioned on a GNN body embedding	MuJoCo; optional Genesis backend	**Tessellated STEP AP242 assembly, tolerance-annotated DXF drawings, interference report, bill of materials, manufacturability score, production archive**	This paper

Table 1 Feature comparison against three widely used evolutionary-robotics platforms. The manufacturing column records what each project's released documentation and repository describe at the time of writing; it does not rule out unpublished extensions. Rows are ordered by breadth of published scope.

3 System overview

ForgeCraft is a layered Python package of 192 modules (approximately 60,000 lines) with a deliberately narrow interface between layers, so that individual research questions can be addressed without modifying the rest of the stack.

3.1 Core morphology representation

The central data structure is `MechanicalBody`, a directed graph of `Part` and `Joint` objects. Parts carry a type and a parameter dictionary, so a body is a typed graph rather than a fixed topology; the representation is memory-conscious by design (`__slots__` on both classes) because populations of thousands of bodies are held in memory during search. Parts are drawn from a *catalogue*: a declarative YAML description of the available part types with their geometric primitive, parameter ranges, mass, friction and actuation properties. Because the catalogue is data rather than code, a new design space is a configuration file rather than a fork; 21 catalogues ship with the release, spanning primitive parts, legged robots, grippers, hoppers and parameterised part libraries.

3.2 Physics evaluation

Evolved bodies are compiled to MuJoCo models [29] in the MuJoCo Extensible Markup Language (XML) format (MJCF) for rigid-body simulation, with an optional Genesis backend for GPU-accelerated evaluation. To keep search tractable, the evaluation path uses deliberately simplified collision geometry (convex proxies), which decouples the cost of physics evaluation from the cost of geometric detail. Environments follow the Gymnasium

API [30], and task specifications — reward composition, episode budget, termination criteria — are themselves declarative YAML.

One modelling detail determines whether the reward terms carry any signal at all: the MJCF builder attaches a `<freejoint/>` to the root body, so every compiled model has a full six-degree-of-freedom floating base. An earlier revision omitted that element and welded the root to the world frame, which made the displacement-, speed- and fall-based reward terms identically zero. Section 6 records the measured before-and-after effect of the repair.

3.3 Search

Co-evolution alternates between population-level search over morphologies and reinforcement-learning updates of the controllers conditioned on those morphologies. Morphologies are encoded by a message-passing graph neural network [31, 32] into fixed-length embeddings, and controllers are PPO [33] actors conditioned on those embeddings. Conditioning the policy on a morphology embedding allows a controller to be trained for an arbitrary topology, but ForgeCraft trains a *separate* controller per evaluated individual within that individual's evaluation budget, optionally warm-started from the previous generation: it does not learn a single policy that generalises zero-shot across body topologies, and we make no such claim. Selection combines elitism, tournament selection and Pareto-based multi-objective selection through the non-dominated sorting genetic algorithms (NSGA) of [34, 35], while a multi-dimensional archive of phenotypic elites [6] maintains behavioural diversity. Search is made affordable by adaptive evaluation budgets that grow with the generation index, and runs support checkpoint and resume.

3.4 Geometry and manufacturing layer

A parametric generator produces concrete mechanical geometry for each abstract part — chassis with ribs and counterbores, motors with cooling fins and end bells, bearings with shields, tubes with threaded ends, springs, feet and battery packs — from the same parameter dictionary used during search. Nine classes of part are generated at four vertex-density presets, and parts receive physically based rendering materials from ten presets. Because search-time bodies and export-time geometry share parameters, the exported artefacts correspond to the design that was actually evaluated. Optional modules provide finite-element and isogeometric analysis, additive-manufacturing pre-processing, knowledge storage and LLM-assisted design agents, isolated behind optional installation extras so that the core pipeline keeps a small dependency footprint; Section 8 records that these analysis modules are incomplete and unused in every result reported here.

4 The manufacturing-aware export chain

The export chain is the contribution claimed in this paper. A single entry point, `export_all_enhanced`, takes an evolved body plus the catalogue it was drawn from and performs the following stages end to end:

1. **Parametric solid generation** for every part in the evolved body.
2. **Connection features** — bolt holes, hinge bearing seats and snap fits generated from the joint type, with clearances derived from fused deposition modelling (FDM) tolerance profiles.
3. **Tolerance and shrinkage compensation** for the selected print profile.
4. **Support and orientation planning**, with support-volume reporting.
5. **Interference checking** between mating part pairs, classifying each fit as clearance, transition or interference.
6. **STEP AP242 assembly export** for CAD interchange, subject to the qualification below.
7. **2D engineering drawings** in DXF with tolerance annotation.
8. **Bill of materials** with mass roll-up and material recommendations.
9. **Manufacturability scoring**, decomposed into build-volume, wall-thickness, assembly-complexity, torque-budget and cantilever checks.
10. **Production packaging** into one archive that can be handed to a fabricator, together with a Unified Robot Description Format (URDF) file for downstream robotics tooling and an optional export of the trained policy to the Open Neural Network Exchange (ONNX) format.

Two properties matter more than the stage list. First, the manufacturability score described in stage 9 is wired into the search layer — a manufacturability term and a minimum-score constraint are both available per task — and Section 7 measures what that wiring currently achieves. Second, the STEP output carries a qualification we will not bury: the exported entities are tessellated (`TESSELLATED_ITEM` with `COORDINATES_LIST` and `TRIANGULATED_FACE`, under the `AP242_MANAGED_MODEL_BASED_3D_ENGINEERING_MIM_LF` schema). The file is valid AP242 and readable by tessellation-aware viewers, but it carries triangle meshes rather than analytic non-uniform rational B-spline (NURBS) surfaces and topological B-rep edges, so it cannot be edited parametrically on

import. A fabricator receiving this package would typically rebuild parametric features from the DXF drawings and the bill of materials rather than from the solid. Closing that gap is future work.

5 Worked example: one complete run

The workflow is exercised by an example committed to the repository (`examples/demo_robot/`), recording the artefacts of a finished evolution run (generation 89, individual 005), started with a single command:

```
python -m forgecraft.main --catalog speedster --task speed --population 32 --generations 100 --seed 42 --export --export-dir design_output
```

The command-line interface (CLI) `--export` flag drives the lightweight export path (stereolithography (STL) and STEP meshes, 3MF, URDF, a bill of materials (BOM) and a review report). The fuller presentation chain, which additionally produces DXF drawings, glTF scenes, rendered views and the packed production archive, is `export_all_enhanced` and is reached through the Python API.

The recorded design comprises 25 parts and 24 joints with a stored composite fitness of 12954.87, dominated by movement terms (`displacement` = 10452.36, `speed` = 2497.80, `upright` = 46.41). The body has eight actuated joints, so the review report no longer emits the "no actuated joints, the body cannot move under its own power" warning that the previous snapshot carried. We nevertheless report the manufacturing diagnostics in full, because they say more about the state of the release than the fitness value does (Table 2, Fig. 1).

Quantity	Value	Source
Parts / joints	25 / 24	`best_body.json`
Composite fitness	12954.87	`evolution_history.json`
Speed component	2497.80	`evolution_history.json`
Displacement component	10452.36	`evolution_history.json`
Upright component	46.41	`evolution_history.json`
Actuated joints	8	`best_body.json`
Colliding part pairs	120	interference check
Collapsed joint ranges	6 of 24	joint-range warnings
Implausible joint ranges	5 of 24 (widest 888.18 rad)	joint-range warnings
Manufacturing aggregates	`overall` 0.9925; score 0.70	`manufacturability.json`

Table 2 Diagnostics for the reference run. The aggregate row records the bookkeeping feature discussed in Section 7.8: the same body is scored by two definitions that answer different questions and are not combined arithmetically.

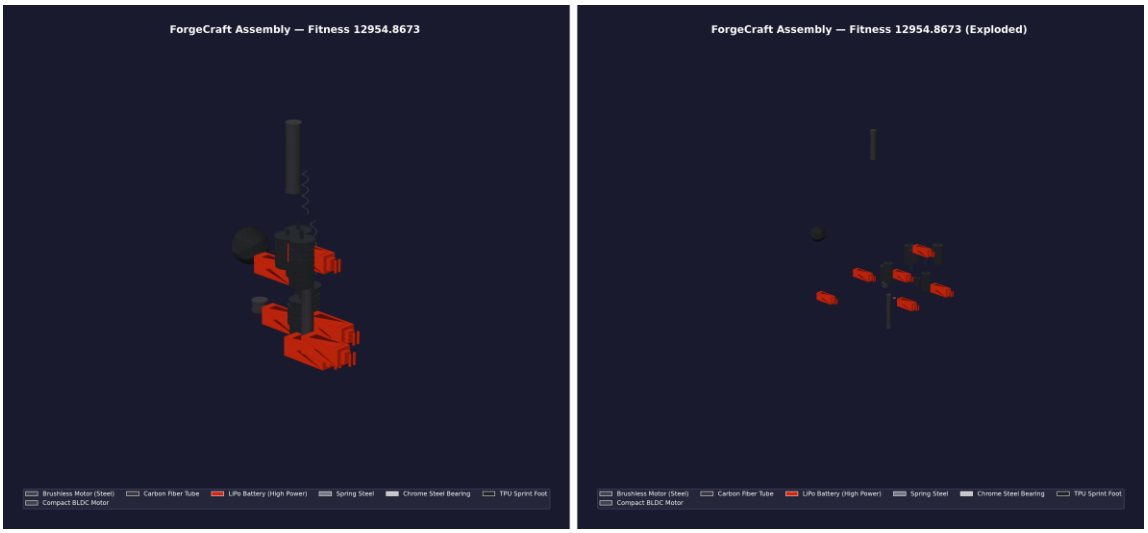


Fig. 1 Assembled (left) and exploded (right) views of the evolved body of Section 5, rendered by the framework's lightweight rasteriser. The 25 parametric parts shown are the ones from which the STEP assembly, DXF drawings and bill of materials are produced. The body is a prototype produced by the pipeline, not a validated design: it contains 120 colliding part pairs and eleven joints with implausible parameter ranges (Table 2).

Three caveats follow, and we state them rather than omit them. First, a stored fitness of 12954.87 is **not** evidence of a high-performing robot: it is a weighted sum dominated by movement terms, reached while the body's parts interpenetrate in 120 pairs and its joints admit physically meaningless travel, neither of which the objective penalises. The honest reading is that the objective now carries a locomotion signal — which, before the repair described in Section 6, it did not — and that self-collision and joint-limit terms must be added before evolved bodies can be read as engineering designs. Second, the aggregate figures disagree: `overall` (0.9925) and `manufacturability_score` (0.70) are two distinct definitions rather than an arithmetic error (Section 7.8), and we report both rather than the more flattering one. We do not report the mass and cost estimates that the same snapshot also carries, because three code paths derive them from three different formulas — bounding-box extents, per-part parameters and a separate price table — and nothing reconciles the results; a single figure could not be defended, and quoting three would imply a precision the models do not have. Third, the committed snapshot is a stored artefact: STL, STEP, DXF, URDF and the packed archive regenerate from `best_body.json` and are deliberately not committed, to keep the repository small.

We therefore present this example as evidence for three narrow claims only: that the pipeline runs end to end from a task specification to a production package, that evolutionary search now receives a locomotion signal, and that the export chain emits inspectable, machine-readable deliverables. It is not evidence that the framework discovered a high-performing robot.

6 Verification and reproducibility

The repository contains 424 automated tests — 253 in the core suite and 171 module-level — covering morphology graph invariants, catalogue loading, genetic operators, selection schemes, checkpoint round-trip, physics model construction, geometry generation, tolerance and connector calculation, interference checking and export formats. Continuous integration runs the suite on Python 3.10–3.12 at every commit, skipping GPU-dependent tests when no Compute Unified Device Architecture (CUDA) device is present. Runs are seeded, task and catalogue definitions are declarative, and checkpoints capture both evolutionary and training state, so an interrupted run resumes to an identical configuration.

The most consequential defect found while preparing this release illustrates why a green test suite is not sufficient evidence. In the previous revision, `forgecraft/simulation/builder.py` emitted the root body without a `<freejoint/>`, so a six-part, five-joint body compiled with no floating base: the centre-of-mass sensor was constant, termination criteria of the form `_com_z < fall_height` never fired, and the `displacement`, `speed` and `velocity` reward terms were identically zero, removing the locomotion signal from the objective altogether. The repair is verified by measurement rather than assertion: the same body now compiles to `nq = 9, nv = 8`, with the free joint at `qposadr = 0` and hinges at `qposadr = 7` and `8`; in a 300-step zero-action rollout the centre of mass descends from 0.476 m to 0.191 m, translates 0.064 m along x and fires the fall termination at step 299 — all exactly zero before the repair. Two latent Gymnasium-contract errors in `forgecraft/rl/env.py` were exposed by the same investigation and are fixed with it: `step()` returned `np.bool_` rather than `bool`, and `reset()` perturbed `qpos` by joint index, which with a floating base corrupts the base position and quaternion. Table 8 in Section 8 collects this defect and the two silent failures of Section 7 into one checklist, and Table 9 maps the module-level suites onto the code they exercise.

7 What a manufacturing term actually does to selection

The framework's design goal is that buildability is optimised rather than filtered. We tested that claim on two instrument builds of the same codebase. The shipped build (tag v0.4.0) returned a clean negative; investigating why exposed two instrumentation defects, which we report and repair. The repaired build (commit a246c14) returns a different answer: the term can reorder a population only to the extent that individuals are separated by fitness gaps smaller than its own contribution, and that extent is measurable in one generation. What the repaired build does not return is a population-level improvement, and Section 7.7 states the replication that would be needed to look for one.

7.1 Removing the term without re-running

Comparing two independently evolved arms is unreliable in this codebase: evaluation runs in worker processes, and on Windows those workers are spawned rather than forked, so they do not inherit the parent seed and two same-seed arms are not strictly paired. We therefore measured *within* a single run. Each individual carries a fitness that includes the manufacturability term and a directly measurable value of that term, so the counterfactual fitness obtained by removing it is `fitness_without = fitness_with − w_mfg · mfg`, with no re-running required.

The effective weight is `w_mfg = 0.05`: the task file lists `manufacturability` among its fitness components but declares no matching reward component, so `get_weight` returns zero and the aggregator's fallback applies (defect D2 in Section 7.3). Every statistic in this section is recomputed from the stored final populations by `recompute_tables.py` and aggregated across seeds by `aggregate_seeds.py` in the ablation experiment directory, which read the same JSON files the runs wrote. One seed is an outlier on the fitness spread and we flag it rather than let it pass: on `primitives` the final population of seed 42 has $\sigma = 1.27$ against a cross-seed median of 0.17, an order of magnitude above every other run, because a single fitness gap above 1.0 inflates the deviation. Every aggregate in this section is therefore a median with its cross-seed range, and where a single population is quoted for illustration it is named as seed 42.

7.2 First measurement (v0.4.0): the term never acted

On the shipped build the term does not change selection. Across 20 generations and a population of 16, removing the term reversed the ordering of no pair of individuals in either catalogue (Table 3). One member of the top five changes identity in `parameterized_parts_v7`, at a boundary that falls inside a three-way fitness tie — the kind of change that cannot alter which individuals selection retains. Repeating the measurement on the repaired build over thirteen seeds changes the conclusion in degree but not in kind: the term is weak rather than absent. The discordant count has a median of 1 of 120 pairs on `primitives` and 3 on `parameterized_parts_v7`, it is non-zero in 7 and 10 of the 13 seeds, and top-five membership changes in 1 and 6 of them. A term that reorders a handful of comparisons in a majority of runs can be measured but cannot be expected to steer a population, which is the question Section 7.7 takes up.

Catalogue	Build	σ(fitness)	Δmfg	r	D / 120	top-5	Seeds D>0
`primitives`	v0.4.0	1.0909	0.0186	0.0009	0 / 120	0	—
`primitives`	repaired	0.17 [0.08–1.27]	0.14 [0.06–0.27]	0.048	1 [0–6]	0	7 / 13
`parameterized_parts_v7`	v0.4.0	0.6043	0.0692	0.0057	0 / 120	0	—
`parameterized_parts_v7`	repaired	0.11 [0.07–0.18]	0.30 [0.19–0.54]	0.132	3 [0–13]	0	10 / 13

Table 3 Counterfactual removal of the manufacturability term, at the deployed weight `w = 0.05` (speed task, population 16, 20 generations). v0.4.0 rows are single runs (seed 42) on `results_{catalogue}.json` at tag `v0.4.0`. Repaired rows aggregate 13 independent seeds: each cell reports the median over seeds, with the cross-seed range in brackets for σ, Δmfg and the discordant count; σ is the population standard deviation of the stored fitness vector and Δmfg the within-population range of the stored score. A discordant pair is a pair whose order genuinely reverses; order changes inside a tie are not counted, because they cannot change which individuals selection keeps. Both v0.4.0 spreads lie below the 0.20 maximum that the shipped scoring function could produce (Section 7.3); the repaired spreads are larger because that is what the repair was for.

A naive rank statistic reads this differently: counting positions after re-sorting gives 2 of 16 individuals moved in `primitives` and 8 of 16 in `parameterized_parts_v7` on the shipped build. Inspection of the sorted fitness values shows why those counts are misleading. The final populations contain only nine distinct fitness values, and 31–38% of individuals sit at exactly 0.0, so a large share of "rank changes" are permutations inside a tie — permutations that selection resolves by identifier, not by score. The discordant-pair count, which excludes ties, is the quantity that answers the question actually being asked, and it is zero on both shipped rows. We report this because the tie structure is itself an independent property of the release: a population in which a third of individuals share the score 0.0 has limited resolution for any added objective term, however well designed.

7.3 Two instrumentation defects

A null result invites the question of whether the instrument was working. It was not, in two independent ways.

D1 — the scoring function had no gradient. The five checks (build volume, wall thickness, assembly complexity, torque budget, cantilever) were each written as a threshold test with a capped penalty. A part either passed, losing nothing, or failed, losing a fixed amount, so the theoretical range of the composite score was [0.80, 1.00] and in practice almost nothing triggered. Sampling 100 randomly generated bodies from each of the 21 shipped catalogues, the score was *identically constant* for 7 of the 21 — no body differed from any other body. A further defect sat inside the torque check: when a body had no actuated joints the check returned 1.0, so a morphology that cannot move under its own power received full marks for its torque budget. Because no combination of checks could move the composite by more than 0.20, no two individuals could ever be separated by a manufacturability contribution larger than $0.05 \times 0.20 = 0.01$, which is why the v0.4.0 rows of Table 3 report spreads of 0.019 and 0.069 and no reordering.

D2 — the weight was never connected. `aggregate_fitness_generic` obtained the manufacturability weight through `get_weight`, which reads `reward.components` — the namespace that configures the reinforcement-

learning reward. The task files declared `manufacturability` in `fitness_components` and `fitness_formula` instead, and `fitness_formula` was used only to compose a description string. The lookup therefore always missed, and the aggregator silently fell back to a hard-coded 0.05. The weights written in the task files — 0.03 for `multi_objective`, 0.01 for `speed_density` — never took effect. This is the more consequential of the two defects, because it is invisible: a declared weight that is silently replaced by a default produces a run that looks correct and is not.

7.4 Repair

Both defects are fixed in the release accompanying this paper.

- All five checks are now graded rather than binary: a design threshold maps to a score of 0.5, with headroom above it and progressive loss below it. The torque check reports *not applicable* for bodies without hinge joints and is dropped from the weighted sum, which is then renormalised over the applicable checks; a body with actuated joints and zero available torque now scores 0.0 rather than 1.0.
- A separate `fitness_weights` namespace is introduced, distinct from the reward configuration. The aggregator resolves weights in the order explicit fitness weight, then reward component, then fallback — and the fallback emits a warning instead of passing silently.
- The internal weights are redistributed according to the resolving power of each check: assembly complexity 0.35, torque budget 0.25, cantilever 0.15, wall thickness 0.15, build volume 0.10.

The effect on the instrument is measurable. Repeating the same sampling procedure (21 catalogues × 100 bodies) gives (Table 4):

Quantity	**v0.4.0**	**Repaired**
Cross-catalogue mean range	0.9760 – 1.0000	0.8147 – 0.9838
Mean range within catalogue	0.0240	0.2254
Catalogues with zero range	7 / 21	0 / 21
`primitives` range	0.0000	0.0958
`parameterized_parts_v7` range	0.0857	0.4627

Table 4 Dynamic range of the fitness-side manufacturability score before and after repair, measured on 100 randomly generated bodies per catalogue for all 21 shipped catalogues. The mean within-catalogue range grows by a factor of 9.4, and no catalogue is constant after the repair.

A gradient, however, is not the same thing as influence, and it is worth being precise about why. The score now discriminates between bodies. Whether that discrimination reaches selection depends on how its spread compares with the spread of the rest of the objective, which the repair does not change.

7.5 A bound on reordering, and the statistic that makes it usable

The bound. Write f for fitness including the term, m for the score, and `g = f − w·m` for the counterfactual. A pair (i, j) reverses only if `sign(f_i − f_j) ≠ sign(g_i − g_j)`; since `g_i − g_j = (f_i − f_j) − w(m_i − m_j)`, that requires `|f_i − f_j| < w·|m_i − m_j| ≤ w·Δmfg`. Reversals are therefore a subset of the pairs whose fitness gap is smaller than the term's maximum contribution. The size of that subset, as a fraction of all pairs, is an upper bound on the discordant fraction. The bound is exact rather than asymptotic, it requires one generation and no counterfactual run, and it is the quantity a practitioner can compute before committing to a weight.

The single-generation statistic. We call that fraction the *at-risk fraction* `A(w)`. It is tighter than the range-based form, because `|m_i − m_j|` is replaced by the actual pairwise difference rather than its maximum: at `w = 0.05` on `primitives`, a median of 22 of 120 pairs fall below `w·Δmfg` but only 15 fall below `w·|m_i − m_j|`. The ratio `r = w·Δmfg/σ(fitness)` orders the two catalogues correctly and is the cheapest summary of the same information, but it is a proxy; the sweep below shows why it should not be read as a threshold.

Catalogue	w	r	At-risk pairs	Discordant	top-5	Seeds with top-5 change
primitives	0.05	0.048	15 / 120	1 [0–6]	0	1 / 13
primitives	0.2	0.192	15 / 120	1 [0–8]	0	1 / 13
primitives	0.5	0.480	20 / 120	3 [0–9]	0	2 / 13
primitives	1	0.959	25 / 120	5 [2–16]	0	5 / 13
primitives	2	1.918	35 / 120	9 [2–20]	2	8 / 13
parameterized_parts_v7	0.05	0.132	19 / 120	3 [0–13]	0	6 / 13
parameterized_parts_v7	0.2	0.528	24 / 120	4 [0–18]	0	6 / 13
parameterized_parts_v7	0.5	1.320	35 / 120	12 [0–24]	0	6 / 13
parameterized_parts_v7	1	2.640	57 / 120	18 [6–32]	2	11 / 13
parameterized_parts_v7	2	5.279	82 / 120	30 [10–56]	4	11 / 13

Table 5 Weight sweep on the repaired build, aggregated over 13 independent seeds. *At-risk pairs* is the number of the 120 possible pairs whose fitness gap is smaller than $w \cdot |mfg_i - mfg_j|$; by the bound above the discordant count can never exceed it, and it does not in any row of any seed. Every cell is the median over seeds with the cross-seed range in brackets; the last column counts the seeds in which top-5 membership changes at all, so a median of zero can hide a change in a minority of runs. Top-5 counts refer to set membership, not to order.

A correction to our own earlier reading. An earlier draft of this section reported a common onset of reordering near $r \approx 0.4$–0.5, inferred from a single seed of each catalogue's sweep. The exact count does not support that reading. primitives already reverses a pair at the deployed weight, where $r \approx 0.048$ — an order of magnitude below the claimed threshold — and yet its median discordant count stays in single digits to $w = 2.0$, while parameterized_parts_v7 reaches double digits only at $w = 0.5$ and above. The ratio orders the two catalogues correctly but does not calibrate them to a shared threshold, because the same σ can describe very different gap structures: both catalogues have a median of eight zero gaps in a population of sixteen, but in one primitives seed a single fitness gap above 1.0 inflates σ, and the median non-zero gap is near 0.04 in both. The at-risk fraction is the quantity with the guarantee; the ratio is a screening summary of it. We state the correction because the superseded claim is precisely the one a reader would carry away. The sweep itself is descriptive: fourteen cells are reported without multiplicity correction, no cell is offered as a significance test, and the only inference drawn from it is the ordering of the bound, which is checked on every cell rather than tested.

Where the bound applies. The inequality holds for any scalarised objective and any weight, so it needs no assumption about the metric's design. Its *usefulness* is narrower: it is informative only while $A(w)$ is small, it evaluates a single generation of the population that selection will actually act on, and it says nothing about steady-state or archive-based selection, where the surviving set is not produced by one ranking. It also cannot say whether a reversal is beneficial — only whether it is possible. It is a screening device for deciding whether a weight is worth running, not a design rule that a weight can be read off.

7.6 Why a weighted sum rather than a third objective

Why the release scalarises. The aggregator composes a scalar fitness, and every search operator in the release consumes that scalar: the generational loop, the NSGA front construction and the phenotypic-elites archive all rank individuals by it. Adding manufacturability as a weighted component is the only change that leaves those operators untouched, which is why the release was built that way. The choice is nevertheless not neutral for the result, and the stored populations say by how much.

Quantifying the alternative. Treating fitness and score as two objectives — the minimal form of the third-objective proposal — leaves 3 of 16 individuals non-dominated on the primitives population, and the most manufacturable individual ($mfg = 1.000$) ranks eighth of sixteen in fitness. A weighted sum at $w = 0.05$ cannot retain that individual; a Pareto rule retains it by construction. On parameterized_parts_v7 the front collapses to a single individual, because the fittest individual happens also to be the most manufacturable. The two catalogues therefore differ in kind and not only in degree: the one in which the weighted sum fails most visibly is also the one in which a Pareto rule would act most strongly.

What this means for the result. Sections 7.2 and 7.7 measure the behaviour of a weighted sum. They do not measure the informativeness of the score, and the distinction matters: under a Pareto rule the term would act on the surviving set with no weight at all, and the replication problem of Section 7.7 would disappear along with the weight. We did not run that comparison and we do not claim it would favour the term — on a sixteen-individual population a two-objective front of three members is itself a small and unstable object. The experiment that would settle it is specific: the same two catalogues and thirteen seeds, `nsga2` with objectives (task fitness, manufacturability, behavioural diversity), reporting the distribution of manufacturability along the final front and the same discordance statistics. Until it is run, the negative result of this section is properly read as a statement about scalarisation.

7.7 Does crossing the bound improve the population?

The bound concerns a single generation. A separate claim would be that choosing a weight large enough to reorder individuals causes evolution to produce more manufacturable populations over many generations. We tested that claim directly, because it does not follow from the bound and it is the claim a reader would actually want.

Using `parameterized_parts_v7` and the speed task, we ran three arms at population 16 for 20 generations over 13 seeds each: the shipped weight `w = 0.05`, the weight that clears the at-risk bound on this catalogue, `w = 0.50`, and a second dose that doubles it, `w = 1.00`. The baseline arm is shared by both comparisons, so the experiment is 26 treatment runs plus 13 baseline runs, and a run at `w = 1.00` takes about thirteen minutes against six at the shipped weight. Because the evaluation workers do not inherit the parent seed (Section 7.1), same-seed arms are not strictly paired, so we report the per-seed differences descriptively and test the arms as independent samples. Table 6 reports the mean manufacturability of the final population together with the ordering statistics computed at each arm's own weight, with a 95% confidence interval (CI) on each difference.

Dose w	Mean mfg	Δ arm means [95% CI]	Up / down	mfg < 0.8	D median	top-5 seeds	r median
0.05 base	0.8099	—	—	88 / 208	3	6 / 13	0.13
0.50	0.7986	-0.0113 [-0.0513, +0.0287]	5 / 8	93 / 208	6	7 / 13	0.65
1.00	0.7710	-0.0389 [-0.0823, +0.0046]	3 / 10	112 / 208	6	6 / 13	0.64

Table 6 Dose response on `parameterized_parts_v7`: 13 seeds per arm, 20 generations, population 16. `Δ mean` is the difference of arm means against the shipped weight with a 95% Welch interval; `seeds up/down` counts per-seed differences against the same-numbered baseline run; `mfg < 0.8` counts individuals below the 0.8 threshold out of 208; `D median` is the median number of discordant pairs out of 120; `top-5 seeds` counts seeds whose top-five set changed; `r median` is the realised diagnostic ratio at that arm's own weight. The two tests on the `w = 1.00` row disagree: the rank test separates the arms at p = 0.048 while the interval on the means contains zero.

The mean does not rise at either dose. At `w = 0.50` the final population averages 0.7986 against 0.8099 for the baseline, a difference of -0.0113 whose Welch interval (-0.0513 to +0.0287, df ≈ 22) contains zero, with 5 seeds moving up against 8 down. At `w = 1.00` the mean is 0.7710, a difference of -0.0389; the interval (-0.0823 to +0.0046) still contains zero, but the rank test returns p = 0.048 and 10 of 13 seeds move against the treatment, so the point estimate leans the other way. The direct contrast between the two doses is -0.0276 (-0.0764 to +0.0212, p = 0.11). The count of individuals below the 0.8 threshold rises monotonically, from 88 of 208 in the baseline to 93 and then 112.

The weight reorders a population; it does not improve one. Computed at each arm's own weight, the median at-risk fraction rises from 0.16 at the shipped weight to 0.19 and 0.22, and the median discordant count from 3 to 6 and 6 of 120 pairs; top-five membership changes in 7 and 6 of 13 seeds. The monotone growth visible in Table 5 comes from re-weighting one fixed population, which by construction must reorder more as the weight rises; across generations that growth does not accumulate. The realised ratio shows why: `r` is 0.13 at the shipped weight, 0.65 at `w = 0.50` and 0.64 at `w = 1.00`. Doubling the weight does not double `r`, because the population adapts: the median fitness spread grows from the 0.109 of Section 7.5 to 0.261 and then 0.657, a sixfold increase, while the median manufacturability spread grows only from 0.303 to 0.349 and 0.461. Most of the added weight is absorbed by a wider fitness distribution rather than converted into a wider manufacturability distribution.

Why 0.50 is the operating point. It is the smallest weight that clears the bound on this catalogue, and it achieves the same ordering statistics as 1.00 while leaving fewer individuals below the threshold (93 against 112 of 208) and a mean that sits 0.028 above the higher dose, though not significantly so. Nothing in these runs supports the assumption that a larger weight buys a more manufacturable population, so the choice should rest on the bound — the criterion that does have support — rather than on an assumed dose-response.

The same calculation states what this experiment can resolve. At `w = 0.50`, with the standard deviation measured on that sample (0.057), thirteen seeds per arm resolve a difference of 0.065 at 80% power, so the interval in Table

6 is consistent with any true gain below about 0.03 and the negative point estimate does not exclude one. Reaching differences of 0.10, 0.07, 0.05 and 0.03 would need 6, 11, 21 and 57 seeds per arm; at `w = 1.00` the wider spread (0.077) raises the floor to 0.089 and those figures to 10, 20, 38 and 105. The seed counts are prospective, for a future experiment; they are not an estimate of the power of the one reported here.

Reducing variance is more economical than adding seeds, and the ordering of remedies is clear from where the variance comes from: make evaluation reproducible and genuinely paired so that a seed identifies a run rather than merely a starting point; run long enough that final populations have converged, so that trajectory stochasticity stops contributing; and only then enlarge the population and add seeds.

7.8 Two different quantities share one name

`compute_manufacturability` applies the five geometric checks described above and is what the search layer consumes. `score_manufacturability` instead scores structure, materials, tolerances and cost, with the structure term driven by the number of colliding part pairs, and is what the exported report quotes. This is the origin of an inconsistency recorded in the repository documentation, where the reference body reports `overall` 0.9925 beside `manufacturability_score` 0.70 in the same file: the two numbers answer different questions. The export-side score is itself close to constant, returning exactly 0.70 for 20 of the 21 catalogues.

7.9 What does work: the export layer

The defects the objective cannot see are detected downstream, and they are the norm rather than the exception. In every run reported here, all 16 individuals in the final population contain colliding part pairs, with up to 151 colliding pairs in a single body, while the fitness-side score of those same bodies remains above 0.97. The reference body of Section 5 carries 120 such pairs and 11 out-of-range joint limits. Self-collision is therefore not a tail case that a filter would catch at the end; it is the typical state of the population, and the interference report is the only component of the framework that surfaces it. We did not compute a rank correlation between the score and the collision count: the run files store the two vectors but not the bodies, so the count would have to be recomputed on a re-run. The qualitative fact is already decisive on its own — a population in which every individual collides cannot be ordered by a score that stays above 0.97 for all of them — and we state the missing correlation rather than substitute an argument for it.

7.10 A screening protocol and its consequence for the contribution

The preceding subsections amount to a decision procedure that costs one evaluated generation and needs no evolutionary run. We state it as a protocol because a negative result is only useful if it is portable: a reader with a different metric, a different population and a different weight should be able to decide in one generation whether the term can matter, instead of spending seeds until the budget is gone.

From the final population of an ordinary run at the deployed weight `w`, compute three quantities. First, the at-risk fraction `A(w)` of Section 7.5; by the bound, no reversal can occur outside that set, so `A(w)` is a ceiling on what the term can do at that weight. Second, the discordant count `D(w)`, the reversals that do occur in `g = f − w·m`; this is what the term does to this population. Third, the number of individuals entering or leaving the top-k set of the current ranking, at the `k = 5` and `k = 10` granularity of Table 5; this is the only one of the three that changes which individuals the next generation is built from. Listing 1 computes all three from the two vectors a run already stores, in `O(n²)` and without an evaluator call.

```
def screen(f, m, w, k):
    pairs = [(i, j) for i in range(len(f)) for j in range(i + 1, len(f))]
    at_risk = sum(1 for i, j in pairs if abs(f[i] - f[j]) < w * abs(m[i] - m[j]))
    g = [fi - w * mi for fi, mi in zip(f, m)]
    disc = sum(1 for i, j in pairs if (f[i] - f[j]) * (g[i] - g[j]) < 0)
    top = lambda v: set(sorted(range(len(v)), key=lambda i: -v[i])[:k])
    return at_risk / len(pairs), disc, len(top(f) ^ top(g))
```

The three quantities separate five cases, and only the last of them justifies a seed budget (Table 7). Each case is instantiated by a row of Table 5, so the protocol is not a hypothetical construction: it re-derives from a single generation what the sweep establishes by enumeration.

Case	Screening output	Reading	Action (row of Table 5)
1	A = 0	no comparison can flip	stop: inert at this weight
2	A > 0, D = 0, top-k same	possible, none realised	stop: no seed budget helps
3	A > 0, D = 0, top-k differs	acts only on exact ties	fix tie resolution first
4	A > 0, D > 0, top-k same	rank churn below the cut	stop: churn, not survival
5	A > 0, D > 0, top-k differs	selection itself changes	proceed; size as Section 7.7

Table 7 Screening decision table. `A(w)` is the at-risk fraction, `D(w)` the count of pairs whose order the term actually reverses, and *top-k* the change in the top-k set of the current ranking. Only the last case can attribute a population-level effect to the weight; the other four say in advance that no such effect exists, and case 3 says that whatever effect there is belongs to tie resolution rather than to manufacture.

Applied to our own runs, the protocol would have changed what we spent. On `primitives` at `w = 0.05` the median at-risk count is 15 of 120 pairs and the median discordant count is 1: in 6 of the 13 seeds no comparison reverses at all, and in 12 the top five is unchanged, so no number of further seeds could have created an effect that the comparisons do not contain. On `parameterized_parts_v7` at the same weight the median is 19 at-risk pairs and 3 discordant ones; the term does act, but in a minority of runs and largely where the population is close to tied — in the seed-42 population it acts only as a tie-breaker, swapping two individuals whose stored fitness agrees to six decimal places (0.245403) while their scores differ by 0.019. On `primitives` at `w = 0.2` a reversal occurs in 9 of the 13 seeds while the top five stays put in 12 of them, which is case 4. At `w = 0.5` on `parameterized_parts_v7` — the first arm Section 7.7 ran — the row reads a median `A = 35/120` and `D = 12` with a top-five median of zero, which is case 4 again: the screen says churn rather than survival, and the run returned exactly that, a mean difference of −0.011 against a cross-seed standard deviation of 0.057. The median row first reaches case 5 on this catalogue at `w = 1.0`, where the top five changes by two members in the typical seed, and we then ran that arm too. Six of the thirteen seeds changed their top five, the median discordant count stayed at six, and the mean fell by 0.039: the screen was right that the higher weight acts on survival, but its projection overshot, because the population it describes adapts and the realised ratio holds near 0.64 instead of rising with the weight. A screening projection is therefore conditional on the spread it was computed from, the more so the further the weight moves from the one that produced the population.

The classification depends on `k`, and the dependence is informative rather than a nuisance. In the seed-42 `primitives` population at `w = 0.05` — case 2 at `k = 5`, since no comparison reverses — applying `k = 3` moves the case to 3, because the two individuals tied at 2.476959 straddle the third rank: the term then changes the elite set without reversing a single comparison. A term whose only action is to choose between bodies the evaluator cannot separate is worth having as a tie-breaker, but it is not worth a seed budget, and the protocol says so at whichever `k` the reader applies it.

The claim this section supports is narrow and, we think, more useful than the one we started with. It is not that manufacturability can be optimised by adding a weight, and not that it cannot; it is that the question is decidable before a run is spent, and that the answer depends jointly on the weight, the metric's spread and the population's gap structure. Three consequences follow. First, a term can be present in the objective, declared in the configuration and reported in every log line, and still be unable to change which individuals survive; Table 7 is the cheapest test of that, and it costs one generation. Second, a metric's dynamic range is a necessary but not a sufficient repair: raising the range from 0.024 to 0.225 made the score informative without making the objective consequential. Third, the choice of scalarisation is itself an experimental variable. The weighted sum was inherited from the aggregator rather than chosen, and Section 7.6 shows that a Pareto rule would retain individuals that no weight at the reported scale can retain. What the framework contributes to manufacturing in this release is therefore the export and verification chain of Section 4 — a design is carried to STEP, DXF, bill of materials and an interference report, and the defects that search ignores are reported there — together with a measurement apparatus for the objective that the search layer consumes. Making the objective term consequential remains open; what is no longer open is how to find out whether it is.

8 Limitations of the current release

Every item below is reproducible from the repository, and each is mirrored in the repository documentation so that the software description and the paper cannot drift apart.

- **The reference body is a prototype, not an engineered design.** It contains 120 colliding part pairs and eleven joints with implausible parameter ranges (six collapsed to [0, 0], five spanning more than 4π rad). Self-collision penalties and physically meaningful joint limits are not yet part of the fitness function.

- **Control training is per-individual.** A separate trainer is constructed for each evaluated individual and updated within that individual's budget; no single policy is shared across bodies and no zero-shot transfer is claimed.
- **Raising the weight reorders a population without improving it.** The bound in Section 7.5 can be cleared by raising the weight, and two doses show what that buys: at weights of 0.50 and 1.00 the final population averages 0.799 and 0.771 against 0.810 at the shipped setting, while the count of individuals below the 0.8 threshold rises from 88 to 93 and 112 of 208. The realised ratio saturates near 0.64 because the population's fitness spread widens sixfold as the weight doubles. Claims about how manufacturability objectives reshape evolution must therefore wait for a cheaper evaluation path, not for a better metric.
- **Population resolution is limited by ties.** Final populations contain about nine distinct fitness values, and 31–38% of individuals score exactly 0.0. Any added objective term competes for differences that the evaluator does not currently resolve, so the at-risk fraction of Section 7.5 is a bound on what a weight can achieve only insofar as the gaps it measures are real rather than quantisation artefacts.
- **The scalarisation was inherited, not chosen.** Manufacturability enters as a weighted component of a scalar fitness because that is what the aggregator and the NSGA front construction consume. Section 7.6 shows that a Pareto treatment would retain individuals no weight at the reported scale can retain, so the negative result reported here is confined to the scalarised formulation until the three-objective run described there is carried out.
- **Geometric decomposition is not manufacturing decomposition.** The parametric layer splits a body into parts that are individually printable and mutually placeable; it does not derive a decomposition from fixturing, tool access, assembly order or tolerance stack-up. A body that passes every check may therefore still require a decomposition a fabricator would not choose.
- **The two manufacturability aggregates coexist without reconciliation.** `overall` and `manufacturability_score` come from independent code paths and are not combined arithmetically; Section 7.8 states what each measures. The mass and cost fields of the export report are withheld from this paper for the same reason: three code paths produce three values for one body, and the discrepancy is unresolved.
- **STEP output is tessellated.** Exports conform to AP242 but carry triangle meshes rather than analytic B-rep geometry (Section 4).
- **Degenerate joint ranges are reported, not rejected.** Range ordering is now normalised in the genetic operators and the joint optimiser, so the reference snapshot contains no inverted range; but nothing constrains how wide a range may become, and the exporter surfaces the resulting warnings rather than refusing to export, so a physically implausible limit reaches the delivered files unless the reader inspects the report.
- **The finite-element solver discretises the bounding box, not the part.** `surface_to_tetrahedralize` in `forgecraft/analysis/fea.py` derives its grid spacing from the part's bounding box and fills that box with regular volume elements; it does not tetrahedralise the part's surface geometry. Results describe a solid block of the part's outer dimensions and must not be read as a stress or safety-factor check.
- **The topology optimiser does not solve a mechanical problem.** The solid isotropic material with penalisation (SIMP) loop in `forgecraft/analysis/topology.py` never assembles or solves Ku = f; `spsolve` is imported but never called, and its sensitivity term depends only on the density field. The `compliance` field of `TopologyResult` is hard-coded to 0.0.
- **Rendering is not physically based.** Figures are rasterised through matplotlib with per-face Lambert/Blinn-Phong/Fresnel shading under a fixed three-light rig — adequate for schematic views, not a path tracer, and not intended for material comparisons.
- **No performance benchmarks are shipped, and distributed evaluation is not exercised in the reported results.** The repository contains no frames-per-second or speed-up measurements; throughput figures from early documentation drafts were removed rather than replaced with estimates. The Ray and multiprocessing paths dispatch to the same rollout-and-PPO worker used in-process and therefore agree with the single-machine path by construction, but the experiments above use the in-process evaluator.
- **No evolved design has been fabricated.** The manufacturing deliverables are generated and internally checked, but no design from this framework has been printed, assembled and tested. This is the largest gap between what the framework produces and what it demonstrates, and it is the next milestone rather than a completed result.

Two of the defects above are worth extracting from the narrative, because neither produces an error, a warning or a failing test: both have the shape of an absence. Table 8 states each as the single assertion that would have failed at the moment it was introduced, which is the form in which a defect of this kind transfers to another codebase.

Defect	Symptom in stored outputs	Assertion that would catch it	Repair
Threshold score	Only 1.0 or 0.0: no gradient	Assert more than two distinct values	Violation count
Wrong config namespace	Silent fallback to the hard-coded 0.05	Assert resolved weight equals config	One config object
No root free joint	Centre of mass flat, rewards zero	Assert `nq` covers the base and moves	Root `<freejoint/>`
Gymnasium drift	`np.bool_` return, `qpos` reset fault	Assert `bool` return and exact reset	Typed returns

Table 8 Silent-failure checklist. None of the four defects raised an exception, emitted a warning or failed a test. The first two are the failures measured in Section 7; the last two were found during the release audit of Section 6. The score returned a constant 1.0 for every passing body and 0.0 for every failing one, so no weight could reorder a population it could not distinguish; the run read its weight from a namespace the task file did not populate and used 0.05 instead of the 0.5 it had been given. The build omitted `<freejoint/>`, leaving the base fixed, and `step` returned `np.bool_` while `reset` perturbed `qpos` by joint index. Each row is the assertion that would have caught the defect at introduction, so the table can be used as a review checklist for a scalarised objective in another framework.

Table 9 records the module boundaries that these assertions protect, together with the size of the suite that covers each one. The counts are the module-level tests in `tests/`; the core suite of Section 6 contributes a further 253 and is not split by module here.

Module	Responsibility	Tests
`forgecraft/llm/`	Providers, knowledge base, five agents, tool registry	55
`forgecraft/evolution/`	Multi-objective, quality-diversity and archive search	54
`forgecraft/core/morphology.py`	Part, joint and body graph invariants	16
`forgecraft/simulation/builder.py`	MJCF assembly: geoms, joint XML, root free joint	14
`forgecraft/evolution/selection.py`	Tournament, dominance, sorting, crowding	9
`forgecraft/rl/env.py`	Gymnasium contract, reward terms, termination	9
`forgecraft/core/catalog.py`	Part-specification loading, default and v2 catalogues	8
`forgecraft/core/generator.py`	Seeded random body generation from a catalogue	6
Total	module-level suite	171

Table 9 Module boundaries and the module-level tests that cover them. A single test file may exercise two packages, which is why `evolution` and the layered nesting code share one row. The table bounds the claim that Section 6 makes: it states what is tested and, by omission, what is not.

9 Discussion

Manufacturability as an experimental variable. The measurement in Section 7 reframes this question rather than closing it. Buildability is reported but not effectively optimised, and the reason is now specific to the point of being quantitative: a term added to a scalar objective can only reorder individuals whose fitness gaps are smaller than its own contribution, so the fraction of such pairs bounds everything any weight can achieve. That fraction had a median of 12–29% for one catalogue across a fortyfold weight sweep and 16–68% for the other, and the difference — not the weight — is what separates the two experiments. The remaining choices are engineering decisions rather than mysteries: compute the at-risk fraction, decide whether the reachable reordering is worth the run, and if not, change the formulation rather than the metric. Once that is done the interesting question becomes available: how does the structure of evolutionary search change when designs must be buildable? Treating part standardisation, minimum feature size and assembly complexity as objectives rather than filters would then turn a post-hoc constraint into an experimental factor. The framework makes this study *possible*; we have not yet carried it out, and Section 7 is an account of why the naive version of it does not work.

Reproducible comparison and a bridge between two communities. By providing a tested, documented and permissively licensed implementation with declarative design spaces, ForgeCraft allows design spaces and tasks to be shared as configuration files, making comparisons between methods concrete rather than approximate. Its output formats target the tools mechanical engineers already use — STEP for CAD interchange, DXF for drawing review, bills of materials for procurement — so an evolved design can enter a mechanical workflow without being re-modelled from scratch, subject to the tessellation caveat above. The full pipeline also runs on commodity laptops with no proprietary software, which supports teaching co-design as an end-to-end engineering activity — from fitness function to drawing sheet — rather than as an isolated optimisation exercise.

10 Conclusion

ForgeCraft couples morphology–control co-evolution with a manufacturing-aware export chain, so that evolved robot designs are delivered together with the geometry, drawings, tolerances and documentation needed to build

them. It integrates established components — graph-based morphology representation, GNN-encoded PPO control, quality-diversity and multi-objective search, MuJoCo physics — and adds an explicit representation of evolved bodies as parametric mechanical parts, together with an export and verification chain that reports the defects the search objective does not see. The paper's contribution is a measurement rather than an algorithm. Measured on two builds of the release, the manufacturing term never acted in the shipped one because the scoring function had no gradient and its weight was never read from the task file; after repair the score discriminates and the term does reorder comparisons, but only in a minority of them, because a scalarised term can only reorder pairs whose fitness gaps are smaller than its own contribution. That condition is an exact bound, computable from a single generation, and it explains why the same nominal weight behaves differently in two catalogues. The three checks that produced this result — a range check on the term's own values, a resolution check on the weight the objective actually receives, and an at-risk screen on one generation — are stated in terms of the objective and the population rather than of this codebase, which is why we present them as an audit rather than as a defect report, with ForgeCraft as the instance in which the first two checks fail. Whether the audit transfers is not something this paper measures: every number here comes from this framework's generation of the codebase and two of its catalogues, and running the screen on a population evolved elsewhere is left open. Clearing the bound does not by itself produce more manufacturable populations: at weights of 0.50 and 1.00 the mean is flat and then lower than at the shipped setting, because the population's fitness spread widens faster than its manufacturability spread. Resolving a difference of 0.10 at the lower dose costs 6 seeds per arm; resolving 0.03 costs 57. ForgeCraft is distributed under the MIT licence with 424 automated tests and continuous integration, and is intended as reusable infrastructure for research at the boundary between evolutionary robotics and design for manufacturing.

Future work is ordered by dependency. First, the search objective needs the terms it currently lacks — self-collision penalties and physically meaningful joint limits — and the mass and cost models must be unified into a single reconciled source before any figure derived from them is reported. Second, the empirical questions the framework exists to answer come into reach: how manufacturability constraints reshape evolved morphologies, and how search behaves when buildability is an objective rather than a filter. Third, and decisively, the loop must be closed in hardware: fabricating and testing an evolved design is the only evidence that will establish whether the manufacturing stage reported here is a real capability or an elaborate serialisation format. We state the scope of this release plainly: it is a research prototype whose export chain is functional and exercised end to end, and whose search chain is now free of the defect that silenced its locomotion signal but still optimises an objective that permits self-intersecting bodies. The reference example is a pipeline-completeness demonstration, not a motile design.

Declarations

Funding. This research received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

Competing interests. The author declares no competing interests.

Author contributions. Zhenxin Liu — Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Data and code availability. The framework is released under the Massachusetts Institute of Technology (MIT) licence at https://github.com/jige0611/forgecraft. The reference run is committed in full under `examples/demo_robot/`; STL, STEP, DXF, URDF and the production archive regenerate from the committed `best_body.json`. The shipped release measured in Section 7.1 is https://github.com/jige0611/forgecraft/releases/tag/v0.4.0; the repair described in Sections 7.2–7.3 and the experiments of Sections 7.4–7.5 are on the default branch of the same repository and are not part of that tag. All experiments were run on Python 3.12.13 with NumPy 2.4.6 and MuJoCo 3.9.0 on a single Windows workstation; `python experiments/ablation_manufacturability/recompute_tables.py` regenerates the per-population statistics of Tables 3 and 5, `aggregate_seeds.py` the cross-seed aggregates of Tables 3, 5 and 6, and `replicate_seeds.py` the seed sweep itself.

Use of generative artificial intelligence (AI). Generative AI tools, specifically the Trae coding assistant, were used for code assistance, experiment scripting and language editing during preparation of this manuscript and the associated software. All scientific claims, results and references were verified by the author, who takes full responsibility for the content.

References

1. Sims K (1994) Evolving virtual creatures. In: Proceedings of the 21st Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '94), pp 15–22. https://doi.org/10.1145/192161.192167
2. Zhao A, Xu J, Konaković-Luković M, Hughes J, Spielberg A, Rus D, Matusik W (2020) RoboGrammar: graph grammar for terrain-optimized robot design. ACM Transactions on Graphics 39(6):188:1–188:16. https://doi.org/10.1145/3414685.3417831
3. Gupta A, Savarese S, Ganguli S, Fei-Fei L (2021) Embodied intelligence via learning and evolution. Nature Communications 12(1):5721. https://doi.org/10.1038/s41467-021-25874-z
4. Yuan Y, Song Y, Luo Z, Sun W, Kitani K. M. (2022) Transform2Act: learning a transform-and-control policy for efficient agent design. In: Proceedings of the 39th International Conference on Machine Learning (ICML), PMLR 162:26992–27015
5. Pathak D, Lu C, Darrell T, Isola P, Efros A. A. (2019) Learning to control self-assembling morphologies: a study of generalization via modularity. In: Advances in Neural Information Processing Systems 32 (NeurIPS)
6. Mouret J-B, Clune J (2015) Illuminating search spaces by mapping elites. arXiv:1504.04909
7. Pugh J. K., Soros L. B., Stanley K. O. (2016) Quality diversity: a new frontier for evolutionary computation. Frontiers in Robotics and AI 3:40. https://doi.org/10.3389/frobt.2016.00040
8. Bhatia J. S., Jackson H, Tian Y, Xu J, Matusik W (2021) Evolution Gym: a large-scale benchmark for evolving soft robots. In: Advances in Neural Information Processing Systems 34 (NeurIPS 2021)
9. Hupkes E, Jelisavcic M, Eiben A. E. (2018) Revolve: a versatile simulator for online robot evolution. In: Applications of Evolutionary Computation (EvoApplications 2018), Lecture Notes in Computer Science 10784. Springer, Cham, pp 687–702. https://doi.org/10.1007/978-3-319-77538-8_46
10. Duarte M, Silva F, Rodrigues T, Oliveira S. M., Christensen A. L. (2014) JBotEvolver: a versatile simulation platform for evolutionary robotics. In: Proceedings of the 14th International Conference on the Synthesis and Simulation of Living Systems (ALIFE). MIT Press, Cambridge, Massachusetts, pp 210–211. https://doi.org/10.7551/978-0-262-32621-6-CH035
11. Gibson I, Rosen D, Stucker B, Khorasani M (2021) Additive Manufacturing Technologies, 3rd edn. Springer, Cham. https://doi.org/10.1007/978-3-030-56127-7
12. Boothroyd G, Dewhurst P, Knight W. A. (2010) Product Design for Manufacture and Assembly, 3rd edn. Taylor & Francis, Boca Raton. https://doi.org/10.1201/9781420089288
13. ISO (2022) ISO 10303-242:2022 Industrial automation systems and integration — product data representation and exchange — part 242: application protocol: managed model-based 3D engineering, 3rd edn. International Organization for Standardization, Geneva
14. ISO (1989) ISO 2768-1:1989 General tolerances — part 1: tolerances for linear and angular dimensions without individual tolerance indications, 1st edn. International Organization for Standardization, Geneva
15. Das I, Dennis J. E. (1997) A closer look at drawbacks of minimizing weighted sums of objectives for Pareto set generation in multicriteria optimization problems. Structural Optimization 14(1):63–69. https://doi.org/10.1007/BF01197559
16. Marler R. T., Arora J. S. (2004) Survey of multi-objective optimization methods for engineering. Structural and Multidisciplinary Optimization 26(6):369–395. https://doi.org/10.1007/s00158-003-0368-6
17. Marler R. T., Arora J. S. (2010) The weighted sum method for multi-objective optimization: new insights. Structural and Multidisciplinary Optimization 41(6):853–862. https://doi.org/10.1007/s00158-009-0460-7
18. Miettinen K (1999) Nonlinear Multiobjective Optimization. Kluwer Academic Publishers, Boston.
19. Zitzler E, Thiele L (1999) Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. IEEE Transactions on Evolutionary Computation 3(4):257–271. https://doi.org/10.1109/4235.797969
20. Zitzler E, Thiele L, Laumanns M, Fonseca C. M., da Fonseca V. G. (2003) Performance assessment of multiobjective optimizers: an analysis and review. IEEE Transactions on Evolutionary Computation 7(2):117–132. https://doi.org/10.1109/tevc.2003.810758
21. Beyer H-G, Sendhoff B (2007) Robust optimization — a comprehensive survey. Computer Methods in Applied Mechanics and Engineering 196(33–34):3190–3218. https://doi.org/10.1016/j.cma.2007.03.003
22. Deb K, Gupta H (2006) Introducing robustness in multi-objective optimization. Evolutionary Computation 14(4):463–494. https://doi.org/10.1162/evco.2006.14.4.463
23. Thompson M. K., Moroni G, Vaneker T, Fadel G, Campbell R. I., Gibson I, Bernard A, Schulz J, Graf P, Ahuja B, Martina F (2016) Design for additive manufacturing: trends, opportunities, considerations, and constraints. CIRP Annals 65(2):737–760. https://doi.org/10.1016/j.cirp.2016.05.004
24. Yang S, Zhao Y. F. (2015) Additive manufacturing-enabled design theory and methodology: a critical review. The International Journal of Advanced Manufacturing Technology 80(1–4):327–342. https://doi.org/10.1007/s00170-015-6994-5
25. Wiberg A, Persson J, Ölvander J (2019) Design for additive manufacturing — a review of available design methods and software. Rapid Prototyping Journal 25(6):1080–1094. https://doi.org/10.1108/rpj-10-2018-0262
26. Knowles J. D., Corne D. W. (2000) Approximating the nondominated front using the Pareto archived evolution strategy. Evolutionary Computation 8(2):149–172. https://doi.org/10.1162/106365600568167
27. Zitzler E, Laumanns M, Thiele L (2001) SPEA2: improving the strength Pareto evolutionary algorithm. Technical report 103, Swiss Federal Institute of Technology (ETH) Zurich. https://doi.org/10.3929/ethz-a-004284029
28. Rosen D. W. (2007) Computer-aided design for additive manufacturing of cellular structures. Computer-Aided Design and Applications 4(5):585–594. https://doi.org/10.1080/16864360.2007.10738493

29. Todorov E, Erez T, Tassa Y (2012) MuJoCo: a physics engine for model-based control. In: International Conference on Intelligent Robots and Systems (IROS), IEEE and the Robotics Society of Japan, pp 5026–5033. https://doi.org/10.1109/IROS.2012.6386109
30. Brockman G, Cheung V, Pettersson L, Schneider J, Schulman J, Tang J, Zaremba W (2016) OpenAI Gym. arXiv:1606.01540
31. Gilmer J, Schoenholz S. S., Riley P. F., Vinyals O, Dahl G. E. (2017) Neural message passing for quantum chemistry. In: Proceedings of the 34th International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research (PMLR) 70:1263–1272
32. Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y (2018) Graph attention networks. In: International Conference on Learning Representations (ICLR 2018). arXiv:1710.10903
33. Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O (2017) Proximal policy optimization algorithms. arXiv:1707.06347
34. Deb K, Pratap A, Agarwal S, Meyarivan T (2002) A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 6(2):182–197. https://doi.org/10.1109/4235.996017
35. Deb K, Jain H (2014) An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. IEEE Transactions on Evolutionary Computation 18(4):577–601. https://doi.org/10.1109/tevc.2013.2281535